PROGRAMMING FOR PROBLEM SOLVING

UNIT 2 Problem Decomposition, Functions and Pointers

PREMANAND S

Assistant Professor

Department of Computational Intelligence

SRM Institute of Science and Technology

Kattankulathur Campus

First Year Engineering

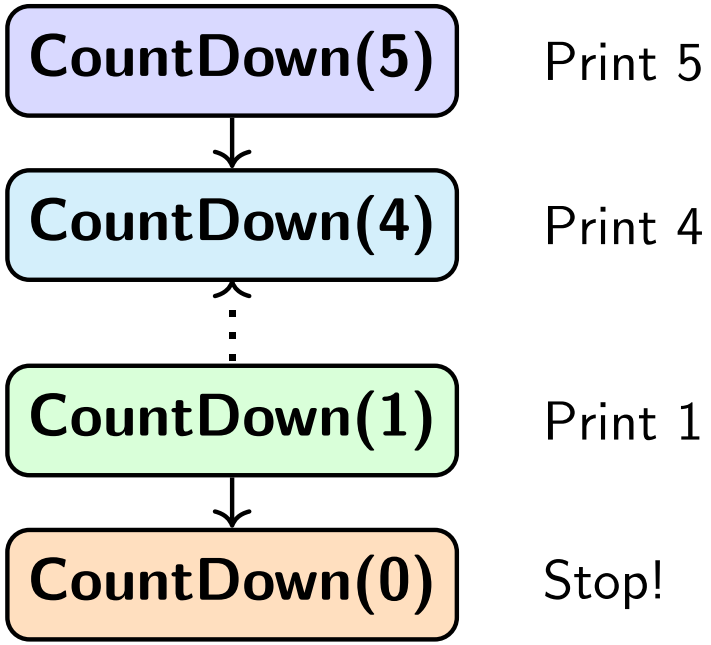
Why Do We Need Recursion?

Imagine This Situation!

"Count down from 5 to 1."

But there is one condition:

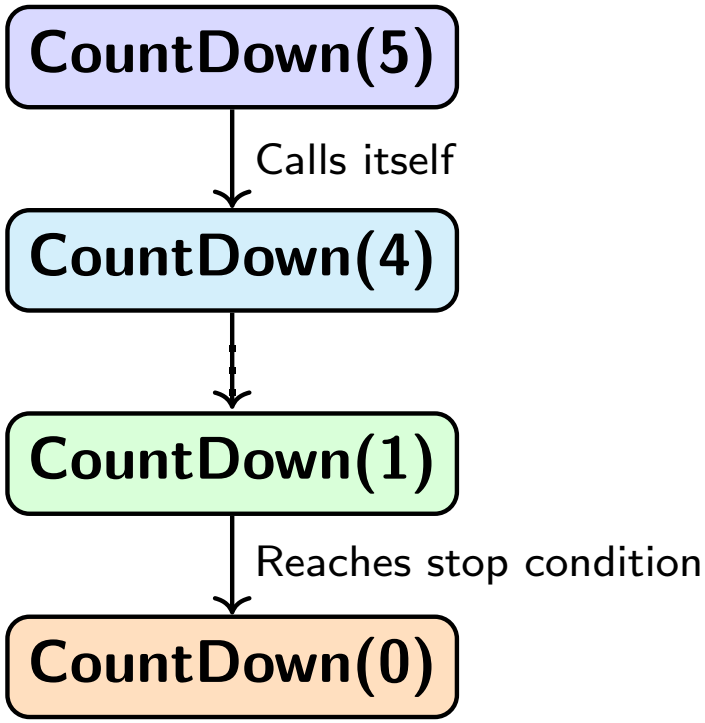
After printing each number, repeat the same task with the next smaller number!



Meet Recursion!

The Big Idea

A function can call itself to solve a smaller version of the same problem.



1. Recursive Call

The function calls itself again with a smaller problem

2. Base Case

The condition that stops further recursive calls

Our First Recursive Function in C

Problem

Print numbers from 5 down to 1 using recursion.

Complete C Program

```
#include <stdio.h>
void countDown(int n);
int main()
{
 countDown(5);
 return 0;  }
void countDown(int n)
{
 if (n == 0)
  return;
 printf("%d ", n);
 countDown(n - 1); }
```

Step 1: Base Case

```
if (n == 0)
 return;
```

When n becomes 0, the function stops.

Step 2: Recursive Call

```
countDown(n - 1);
```

The function calls itself with a

What Happens When Recursive Call Returns?

Remember: When `n == 0`, the `return` statement stops that function call.
Control then returns to the previous call.

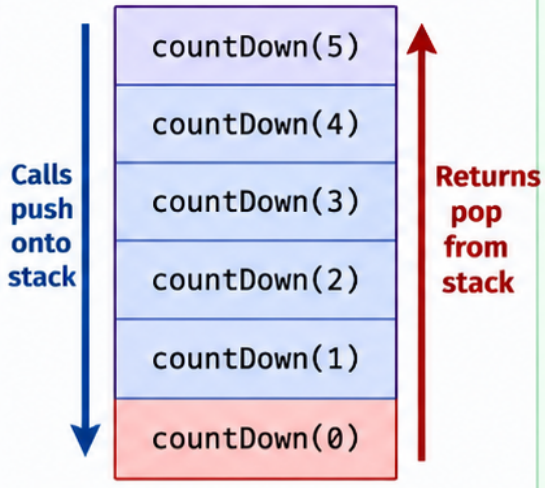
Going Down

Recursive Calls (towards base case)

`countDown(5)`	`n = 5` (prints 5)
↓	
`countDown(4)`	`n = 4` (prints 4)
↓	
`countDown(3)`	`n = 3` (prints 3)
↓	
`countDown(2)`	`n = 2` (prints 2)
↓	
`countDown(1)`	`n = 1` (prints 1)
↓	
`countDown(0)`	`n = 0` **Base case reached!** No further call.

Call Stack

(During recursive calls)



Coming Back

Returning from Calls (after base case)

- `countDown(5)` finishes
- `countDown(4)` finishes
- `countDown(3)` finishes
- `countDown(2)` finishes
- `countDown(1)` finishes
- `countDown(0)` returns

Output of the Program

```
5 4 3 2 1
```

Key Takeaway

- Each function call waits for the next call to finish.
- When the base case is reached, the calls return one by one.
- In this program, nothing is printed during the return journey because there are no statements after the recursive call.

Recursion Practice – Predict the Output

Question

Printing Numbers in Ascending Order

Recursion Practice – Predict the Output

C Program

```
#include <stdio.h>
void printNumbers(int n);
int main()
{
 int n;
 printf("Enter a number: ");
 scanf("%d", &n);
 printNumbers(n);
 return 0;
}
```

Recursion Practice – Predict the Output

C Program

```
void printNumbers(int n)
{
 if (n == 0)
 return;
 printNumbers(n - 1);
 printf("%d ", n);
}
```

Tasks

1. What is the output if the user enters 5?
2. What is the purpose of `if (n == 0)`?
3. Why is `printf()` placed after the recursive call?

Recursion Practice – Predict the Output

Question

Sum of the First N Natural Numbers Using Recursion

Recursion Practice – Predict the Output

C Program

```
#include <stdio.h>
int sum(int n);
int main()
{
 int n, result;
 printf("Enter a number: ");
 scanf("%d", &n);
 result = sum(n);
 printf("Sum = %d", result);
 return 0;
}
```

Recursion Practice – Predict the Output

C Program

```
int sum(int n)
{
 if (n == 0)
 return 0;

 return n + sum(n - 1);
}
```

Tasks

1. What is the output if the user enters 5?
2. What is the purpose of `if (n == 0)`?
3. How does `return n + sum(n - 1)` calculate the sum?

Recursion Practice – Predict the Output

Question

Fibonacci Using Recursion

Recursion Practice – Predict the Output

C Program

```
#include <stdio.h>
int fibonacci(int n);
int main()
{
 int n, result;
 printf("Enter the position N: ");
 scanf("%d", &n);
 if (n < 0)
 {
  printf("Enter a non-negative integer.");
  return 0;    }
 result = fibonacci(n);
 printf("Fibonacci number at position %d = %d",
 n, result);
 return 0;    }
```

Recursion Practice – Predict the Output

C Program

```
int fibonacci(int n)
{
 if (n == 0)
  return 0;
 if (n == 1)
  return 1;
 return fibonacci(n - 1) + fibonacci(n - 2);
}
```

Tasks

1. What is the output if the user enters 6?
2. What is the purpose of the two base cases?
3. Why does the function make two recursive calls?

Recursion Practice – Predict the Output

Question

GCD and LCM Using Recursion

Recursion Practice – Predict the Output

C Program

```
#include <stdio.h>
int gcd(int a, int b);
int main()
{
 int a, b, g, lcm;
 printf("Enter two positive integers: ");
 scanf("%d %d", &a, &b);
 if (a <= 0 || b <= 0)
 {
  printf("Please enter positive integers.");
  return 0;  }
 g = gcd(a, b);
 lcm = (a / g) * b;
 printf("GCD = %d\n", g);
 printf("LCM = %d", lcm);
 return 0;  }
```

Recursion Practice – Predict the Output

C Program

```
int gcd(int a, int b)
{
 if (b == 0)
 return a;
 return gcd(b, a % b);
}
```

Tasks

1. What is the output if the user enters `12 18`?
2. What is the purpose of `if (b == 0)`?
3. How does `gcd(b, a % b)` help find the GCD?
4. How is the LCM calculated using the GCD?

A Function That Cannot Change the Original?

Imagine This Situation!

You have Rs. 500 in your wallet. You ask your friend to increase your balance by Rs. 100.
Your friend changes a copy of your balance to Rs. 600. But your actual wallet still contains Rs. 500!

Pass by Value

Rs. 500

A copy is changed to Rs. 600.

Original remains Rs. 500

Our Goal

Rs. 600

We want to change the actual balance.

Original becomes Rs. 600

How can a function change the original variable?

Let's Test What We Already Know!

Problem

Can a function change the original variable when we pass its value as an argument?

C Program

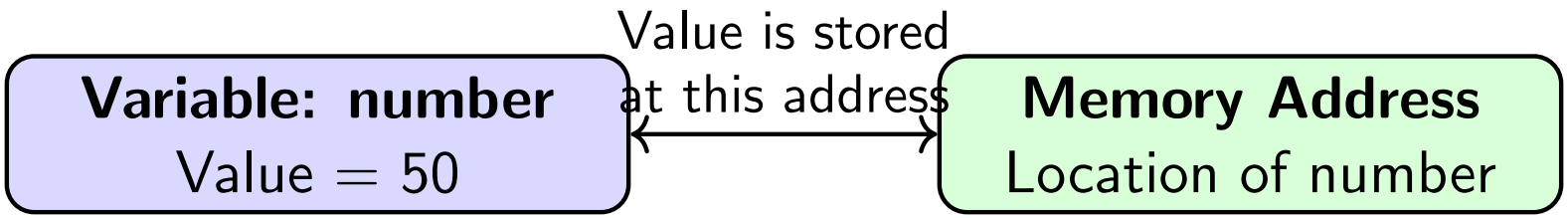
```
#include <stdio.h>
void change(int x);
int main()
{
 int number = 50;
 change(number);
 printf("Original number = %d", number);
 return 0;   }
void change(int x)
{
 x = 100;   }
```

What If We Give the Function the Address?

Recall What We Learned About Pointers

Every variable has two important things:

- **Value** – the data stored in the variable.
- **Address** – the location where the variable is stored in memory.



Two Different Ways

- `change(number)` – passes the value 50.
- `change(&number)` – passes the address of `number`.

Think About It!

If a function receives the address of the original variable, can it access and modify that variable?

Passing the Address to a Function

Step 1: Pass the Address

Instead of passing the value, pass the address of the original variable.

Function Call

```
int number = 50;

change(&number);
```

- `&number` means the address of `number`.
- The function receives this address through a pointer.

Passing the Address to a Function

Step 2: Receive the Address

```
void change(int *p)
{
 *p = 100;
}
```

- `int *p` declares a pointer parameter.
- `*p = 100` changes the value at the stored address.

Address passed $\longrightarrow$ Original value modified

How Does the Original Value Change?

Step 1: The Original Variable

```
int number = 50;
```

The variable `number` stores the value **50**.

Step 2: Pass the Address

```
change(&number);
```

The address of `number` is passed to the function.

How Does the Original Value Change?

Step 3: Receive and Modify

```
void change(int *p)
    *p = 100;
```

The pointer `p` refers to `number`. Therefore, `*p = 100` changes the original value.

Step 4: Back in main()

```
printf("%d", number);
```

Output: 100

Complete Program: Modify the Original Variable

C Program

```
#include <stdio.h>
void change(int *p);
int main()
{
 int number = 50;
 printf("Before: %d\n", number);
 change(&number);
 printf("After: %d\n", number);
 return 0;
}
void change(int *p)
{
 *p = 100;
}
```

Pass by Value vs. Pass by Reference Using Pointers

Feature	**Pass by Value**	**Pointer-Based Passing**
Function parameter	`int x`	`int *p`
Function call	`change(number)`	`change(&number)`
What is passed?	Copy of the value	Address of the variable
How to access data?	Use `x`	Use `*p`
Can the original change?	No, not by modifying the parameter	Yes, through the pointer
Example	`x = 100;`	`*p = 100;`

Remember

Need only the value? Use **pass by value**.

Need to modify the original variable? Pass its **address using a pointer**.